

Báo cáo tiến độ đề án: HermesQA — Bot AI tự động review Pull Request

Cập nhật: 07/10/2026 · **Trạng thái:** Hệ thống chạy đầu-cuối; thực nghiệm đã chạy đủ 2 bộ dữ liệu + ablation self-verify (mục 7.1–7.5, số liệu chốt với pipeline cũ); đã triển khai 7 cải tiến rút ra từ khảo sát Open Code Review (mục 5 nhóm E); **đã chạy lại xong thực nghiệm với pipeline mới** (mục 7.6: 35/35 và 18/18 lượt, F1 11m 89,5 · both 87,2) và **pilot trên 15 PR thật** cho kết quả âm (mục 7.6 cuối)

1. Bài toán và mục tiêu

1.1 Vấn đề

Code review thủ công là nút thắt trong quy trình phát triển phần mềm: tốn thời gian của lập trình viên giàu kinh nghiệm, chất lượng không đồng đều giữa các reviewer, và người review dễ bỏ sót lỗi bảo mật khi đang tập trung vào logic nghiệp vụ.

Hai hướng tự động hóa hiện có đều không trọn vẹn: - **Static analysis** (semgrep, SonarQube...): chính xác, rẻ, nhưng chỉ bắt được các mẫu lỗi đã định nghĩa sẵn; không hiểu ngữ cảnh nghiệp vụ, không biết "thay đổi này thiếu test cho trường hợp nào". - **LLM review thuần túy**: hiểu ngữ cảnh, nhận xét được về thiết kế và test, nhưng dễ *hallucinate* (bịa lỗi không tồn tại, báo sai số dòng) và tốn chi phí token.

1.2 Mục tiêu đề án

Xây dựng hệ thống kết hợp cả hai, với ba đóng góp chính:

- Kiến trúc đa vai trò (multi-role)**: mô phỏng quy trình review thật với ba vai **SE** (Software Engineer — logic, thiết kế), **QA** (test coverage, edge case), **DevOps** (Dockerfile, CI, hạ tầng). Mỗi vai có một *skill prompt* chuyên sâu riêng.
- Sandbox an toàn**: code trong PR đến từ nguồn không tin cậy, nên static tools chạy trong container Docker cô lập hoàn toàn (không có mạng, bỏ mọi capability, source mount chỉ đọc). **Code của PR không bao giờ được thực thi.**
- Cơ chế chống hallucination**: phần mềm *ghim* mỗi nhận xét vào đúng dòng bằng đoạn code LLM chép nguyên văn (không tin số dòng LLM tự đếm), bước *kiểm tra sự thật* (LLM chỉ được loại nhận xét khi nêu được bằng chứng mà phần mềm kiểm được), checklist theo loại file có mục "không báo", dedupe với kết quả static tool, lọc theo confidence và ràng buộc số dòng phải nằm trong diff. Phiên bản đầu dùng *self-verify* (LLM tự kiểm chứng chính mình); thực nghiệm cho thấy bước này không có lợi ích đo được nên đã được thay (mục 4, mục 5 nhóm E).

2. Kiến trúc hệ thống

GitHub webhook → FastAPI (api) → Redis queue → Worker

- └ clone PR head (depth 1)
- └ sandbox: static tools (--network none)
- └ router: chọn vai theo file thay đổi
- └ pipeline (dùng chung với CLI):
 - | chọn file (chặn secret/generated) → che dòng bí mật
 - | → chia file theo vai, chia lô → Agent x vai
 - | (SKILL.md + checklist theo loại file)
 - | → ghim bằng existing_code → fact-check
- └ dedupe / confidence gate / max comments / dấu vân tay
- └ PR review + Checks API + report .md + Postgres

2.1 Vì sao tách API và Worker qua hàng đợi?

GitHub yêu cầu webhook phản hồi trong **10 giây**, trong khi một lần review mất 1–3 phút. Vì vậy tầng API chỉ xác thực chữ ký HMAC, đẩy job vào Redis rồi trả `202` ngay lập tức. Worker xử lý bất đồng bộ. Đây là ràng buộc bắt buộc của nền tảng, không phải lựa chọn tối ưu hóa.

2.2 Vai trò từng module (~2.000 dòng Python trong `app/`, thêm ~2.350 dòng eval và test)

File	Trách nhiệm
<code>app/main.py</code>	Webhook FastAPI: xác thực HMAC SHA-256, lọc sự kiện, đẩy job vào queue
<code>app/worker.py</code>	Pipeline review đầy đủ cho 1 PR (7 bước, xem mục 2.3); phần LLM gọi <code>pipeline.py</code>
<code>app/cli.py</code>	Chạy offline trên repo local — dùng cho phát triển và chạy thực nghiệm; phần LLM gọi cùng <code>pipeline.py</code> với worker
<code>app/github_app.py</code>	Xác thực GitHub App (JWT → installation token), lấy diff, đăng review, Checks API
<code>app/sandbox.py</code>	Khởi chạy container static analysis với các ràng buộc an toàn
<code>app/memory.py</code>	Lưu lịch sử review, "bộ nhớ repo" (convention team đã thống nhất) và dấu vân tay comment đã đăng vào Postgres
<code>app/review/pipeline.py</code>	Mới. Phần LLM của pipeline, dùng chung cho CLI và worker: chọn file → che bí mật → chia lô → gọi agent → ghim → fact-check. Gom về một chỗ để "đường eval đo" và "đường người dùng đi" là một (bài học từ bug D3)
<code>app/review/selection.py</code>	Mới. Phần tắt định trước khi gọi LLM: chặn file bí mật, bỏ file sinh tự động, chia file theo vai, chia lô theo ngân sách ký tự; mọi lần bỏ/cắt đều được trả về để ghi vào report
<code>app/review/rules.py</code> + <code>app/skills/rules/*.md</code>	Mới. Checklist theo loại file (Python, JS/TS, Go, Java, Dockerfile, CI, manifest dependency, file test, mặc định), mỗi checklist có mục "Không báo" ; ghép vào prompt theo đúng loại file trong lô
<code>app/review/agent.py</code>	Backend LLM (OpenAI-compatible, Anthropic, Hermes CLI); bước fact-check (thay self-verify); bản self-verify cũ giữ lại làm đối chứng (<code>--verify legacy</code>)
<code>app/review/diff_utils.py</code>	Parse unified diff, ánh xạ số dòng để comment inline, trích ngữ cảnh file; resolve_anchor ghim finding bằng đoạn code nguyên văn; che dòng bí mật khi render patch
<code>app/review/router.py</code>	Chọn vai nào cần chạy dựa trên loại file thay đổi; PR chỉ sửa file test giao cho QA
<code>app/review/static_tools.py</code>	Chuẩn hóa output 5 công cụ về cùng một cấu trúc <code>Finding</code> ; bộ lọc nhiễu có tài liệu (B101 trong test)
<code>app/review/postprocess.py</code>	Dedupe nhận biết nguồn, áp policy, định dạng comment, quyết định kết luận check run; dấu vân tay comment
<code>app/review/report.py</code>	Sinh báo cáo Markdown (có mục "Phạm vi review" và danh sách nhận xét bị loại sau fact-check) và phần tóm tắt đăng lên PR
<code>app/skills/{SE,QA,DevOps}/SKILL.md</code>	Prompt chuyên sâu cho từng vai (checklist, thang severity, quy tắc "chỉ báo vấn đề do diff gây ra")

2.3 Bảy bước của pipeline review

- 1. **Clone** PR head với `--depth 1` , sau đó **gỡ remote ngay** để token không nằm lại trong `.git` .
- 2. **Đọc cấu hình** `.hermesqa.yml` của repo (chọn vai, ignore glob, convention, ngưỡng chặn merge).
- 3. **Static analysis** trong sandbox; chỉ giữ finding thuộc file có trong diff. Static quét **cả** file có thể chứa bí mật (sandbox không có mạng), chỉ LLM mới không được thấy chúng.

4. **Chuẩn bị đầu vào cho LLM — phần tắt định, không giao cho model** (`selection.py`, dùng chung cho CLI và worker qua `pipeline.py`): - loại file có thể chứa bí mật (`.env`, khoá riêng, thư mục `secrets/...`) và file sinh tự động (lockfile, `*.min.js`, `vendor/...`); cổng chặn bí mật đứng trước mọi cấu hình của người dùng, không có cách bật lại; - che những dòng mà gitleaks đã đánh dấu là bí mật trước khi đưa diff vào prompt; - mỗi vai chỉ nhận file thuộc chuyên môn (SE: code không phải test; QA: code + test; DevOps: Dockerfile/CI/manifest); - chia lô theo ngân sách ký tự thay vì cắt im lặng; file quá lớn bị cắt thì việc cắt được **ghi vào report**.
5. **Chạy agent** theo từng vai, từng lô; prompt gồm diff + ngữ cảnh file + tóm tắt kết quả static (để *không lặp lại*) + bộ nhớ repo + **checklist theo loại file** có mục "Không báo" (`rules.py`, `app/skills/rules/*.md`). Mỗi finding phải chép nguyên văn 1–3 dòng code (`existing_code`); phần mềm **ghim** finding vào dòng thật bằng so khớp chuỗi (`resolve_anchor`), không tin số dòng LLM tự đếm.
6. **Fact-check**: LLM chỉ được *đề nghị* loại một finding khi nêu căn cứ A (code được mô tả không tồn tại) hoặc B (một dòng cụ thể phủ định nhận xét); phần mềm kiểm căn cứ đó trên diff trước khi loại. Mặc định là giữ; security/bug mức *high* trở lên không bao giờ bị loại. Finding bị loại được liệt kê trong report kèm lý do.
7. **Post-process và đăng kết quả**: dedupe nhận biết nguồn, lọc theo confidence/severity, giới hạn số comment; tính **dấu vân tay** cho từng comment (file + loại + nội dung dòng code, không dùng số dòng) để lần push sau không đăng lại comment cũ; comment inline lên PR, cập nhật GitHub Checks, sinh báo cáo Markdown có mục "Phạm vi review", lưu Postgres.

2.4 Bộ công cụ static analysis (5 công cụ)

Công cụ	Phạm vi
semgrep	Mẫu lỗi bảo mật/logic đa ngôn ngữ (SQL injection, command injection...)
bandit	Lỗi bảo mật đặc thù Python (<code>shell=True</code> , <code>eval</code> , MD5...)
gitleaks	Secret/API key bị hard-code
hadolint	Lỗi và anti-pattern trong Dockerfile
ruff	Style, import thừa, lỗi cú pháp Python

3. Thiết kế bảo mật

Đây là phần cần nhấn mạnh khi bảo vệ, vì hệ thống **nhận và xử lý code từ người lạ** (bất kỳ ai mở PR).

Rủi ro	Biện pháp
Code độc hại trong PR được thực thi	Không bao giờ thực thi code PR. Chỉ static tools đọc file.
Công cụ phân tích bị lợi dụng để gọi ra ngoài	Container chạy với <code>--network none</code> — không có mạng
Leo thang đặc quyền trong container	<code>cap_drop: ALL</code> , <code>security_opt: no-new-privileges</code>
Cạn kiệt tài nguyên (fork bomb, quét vô hạn)	Giới hạn 2 CPU, 2 GB RAM, <code>pids_limit=256</code> , timeout 600s
Sửa đổi mã nguồn khi phân tích	Mount <code>/src</code> ở chế độ read-only
Lộ token GitHub	Dùng installation token ngắn hạn; gỡ remote khỏi <code>.git</code> ngay sau clone
Webhook giả mạo	Xác thực HMAC SHA-256 với <code>compare_digest</code> (chống timing attack)

Hệ quả thiết kế đáng lưu ý: vì sandbox không có mạng, bộ rule của semgrep phải được **nạp sẵn vào image lúc build** (`/opt/semgrep-rules`) thay vì tải lúc chạy bằng `--config auto`. Đây là một lỗi thực tế đã gặp và sửa

4. Cơ chế chống hallucination (đóng góp học thuật chính)

LLM khi review code thường mắc ba lỗi: bịa ra vấn đề không tồn tại, báo sai số dòng, và lặp lại điều static tool đã nói. Nguyên tắc thiết kế rút ra sau khi khảo sát Open Code Review của Alibaba ([docs/DE_XUAT_CAI_TIEN_TU_OPEN_CODE_REVIEW.md](#)): **việc gì làm được bằng quy tắc cứng thì không giao cho model**. Hệ thống dùng **sáu lớp phòng vệ xếp chồng**, trong đó bốn lớp là phần mềm tất định:

1. **Ràng buộc đầu vào**: prompt đưa kèm số dòng thật của từng dòng code; checklist theo loại file (lớp 6) nhắc "chỉ báo vấn đề do thay đổi này gây ra".
2. **Ghim bằng đoạn code nguyên văn (anchor)** — *thay cho việc tin số dòng*: mỗi finding phải chép lại 1–3 dòng code (`existing_code`). Phần mềm tìm đoạn đó trong diff bằng so khớp chuỗi (bỏ qua khoảng trắng, bỏ dấu `+/ -` LLM hay chép theo, khớp dòng dài nhất nếu chép thiếu), ưu tiên dòng `+` và dòng gần số LLM đưa nhất. LLM hiểu code tốt nhưng *đếm dòng kém*; chép lại code thì bền hơn nhiều. Finding không ghim được mới quay về số dòng LLM đưa, và cách ghim (`code` / `line` / `snap` / `none`) được ghi lại để eval đo. Ở lần chạy chốt (07/10, `eval/out-v3`), **39/39 finding của 11m ghim được bằng code** — và F1 ở tolerance 0 bằng đúng F1 ở ± 3 dòng (89,5 %), nghĩa là không còn finding nào phải nhờ sai số dòng mới khớp (mục 7.6).
3. **Fact-check** — *thay cho self-verify*: bản cũ đưa toàn bộ finding cho LLM "reviewer cấp cao" sinh lại danh sách đã lọc; thực nghiệm (mục 7.4, điểm 4) cho thấy nó không lọc được gì mà tốn gấp đôi token, và vì sinh lại nên có thể làm hỏng finding đúng. Bản mới đảo mặc định: **giữ mọi finding**, LLM chỉ được *đề nghị* loại khi nêu đúng một trong hai căn cứ — **A**: đoạn code nhận xét mô tả không tồn tại trong diff/context; **B**: một dòng cụ thể trong diff phủ định trực tiếp nhận xét, phải chép nguyên văn. Sau đó **phần mềm kiểm căn cứ**: với A, `existing_code` phải thật sự không tìm thấy; với B, dòng bằng chứng phải có thật trong diff **và không được trùng chính dòng bị báo** (lý luận vòng: "nhận xét nói dùng `== None`, nhưng dòng 12 đã có `if user == None`" — dòng 12 chính là chỗ sai; lần đo 05/10 cho thấy model hạng nhẹ loại 4/5 finding đúng theo kiểu này, xem E8). Security/bug mức *high* trở lên không bao giờ bị loại. Mọi finding bị loại đi vào report kèm căn cứ, để người đọc kiểm tra lại được.
4. **Ràng buộc kỹ thuật khi map dòng**: `commentable_line()` chỉ chấp nhận dòng nằm trong diff; nếu lệch ≤ 3 dòng thì "snap" về dòng gần nhất, lệch hơn thì finding bị chuyển sang phần phụ lục thay vì comment inline.
5. **Gộp kết quả nhận biết nguồn (dedupe)**: hai finding cách nhau ≤ 2 dòng trong cùng file được coi là cùng vị trí. Giữa static tool và LLM, cùng vị trí là đủ để gộp (hai bên dùng hai taxonomy khác nhau cho cùng một lỗi: ruff gọi `== None` là *bug*, LLM gọi là *maintainability*), **với điều kiện** finding LLM không nghiêm trọng hơn finding static; nếu LLM cho rằng chỗ đó có lỗi nặng hơn (semgrep *medium* cạnh nil-deref *critical*) thì đó là hai lỗi khác nhau, giữ cả hai. Giữa hai vai LLM với nhau vẫn đòi cùng category, vì SE báo *bug* và QA báo *thiếu test* cho cùng hàm là hai nhận xét khác nhau. Finding chính là static tool (mô tả tất định); finding bị gộp được **đính kèm vào comment** chứ không vớt bỏ, và severity của cụm lấy mức cao nhất. Quy tắc này là kết quả của thực nghiệm (mục 7.4): bản đầu đòi cùng category giữa mọi nguồn nên gần như không gộp được gì.
6. **Checklist theo loại file có mục "Không báo", và chia file theo vai**: SKILL.md của vai trả lời "vai này quan tâm gì"; checklist (`app/skills/rules/*.md`, cấu trúc phòng theo `rule_docs` của Open Code Review, Apache-2.0) trả lời "với loại file này, báo gì và không báo gì". Ví dụ Dockerfile: *không báo* thiếu `HEALTHCHECK`, thiếu multi-stage, chưa ghim digest khi diff không đụng tới phần đó; Python: *không báo* "thiếu đóng kết nối" ở code cũ, "thiếu validation" chung chung; file test: chỉ báo test bị skip/xoá/làm yếu assertion. Cùng lúc, mỗi vai chỉ nhận file thuộc chuyên môn (SE không nhận Dockerfile, DevOps không nhận logic, QA không đòi

test cho file cấu hình) và "thiếu unit test" chỉ được báo cho **hàm mới**. Ba nguồn nhiễu này chiếm phần lớn FP đo được ở mục 7.4 (điểm 3). Rủi ro của lớp này là *overfit* checklist vào bộ eval tự tạo — xem mục 9.

Ngoài ra còn **confidence gate** (mặc định $\geq 0,6$), **giới hạn số comment** (mặc định 15) và **dấu vân tay comment** (file + loại + nội dung dòng code; không dùng số dòng vì trôi khi push thêm, không dùng tiêu đề vì LLM mỗi lần diễn đạt một kiểu) để lần push sau không đăng lại nhận xét cũ — kết luận chặn/không chặn merge vẫn tính trên toàn bộ.

5. Các lỗi đã phát hiện và sửa trong quá trình triển khai

Phần này minh chứng cho việc hệ thống đã thực sự được chạy và kiểm thử, không chỉ viết xong để đó.

Nhóm A — Lỗi khiến chức năng không chạy được

#	Lỗi	Nguyên nhân gốc	Cách sửa
A1	Semgrep chưa bao giờ chạy được	Sandbox <code>--network none</code> nhưng dùng <code>--config auto</code> (cần tải rule từ Internet)	Nạp sẵn ruleset vào image lúc build
A2	Semgrep crash ngay khi khởi động	<code>python:3.12-slim</code> không còn <code>pkg_resources</code> ; semgrep 1.90 vẫn phụ thuộc	Ghim <code>setuptools<81</code> trong image
A3	Semgrep từ chối toàn bộ thư mục rule	Repo <code>semgrep-rules</code> chứa <code>.pre-commit-config.yaml</code> không phải file rule	Xóa các YAML không phải rule lúc build
A4	Bandit được cài nhưng không bao giờ được gọi	Thiếu lệnh gọi trong script runner và thiếu parser	Bổ sung lệnh gọi + parser chuẩn hóa output
A5	Bước self-verify chưa bao giờ chạy được	<code>VERIFY_PROMPT</code> ghép thêm <code>OUTPUT_CONTRACT</code> (chứa <code>{}</code> của JSON schema) rồi gọi <code>.format()</code> → Python hiểu nhầm <code>{"summary"...}</code> là placeholder → <code>KeyError</code> . Lỗi bị <code>try/except</code> nuốt mất nên không ai biết.	Ghép <code>OUTPUT_CONTRACT</code> sau khi format; bổ sung test hồi quy

A5 là lỗi nghiêm trọng nhất: cơ chế chống hallucination — đóng góp chính của đề án — thực chất chưa từng hoạt động, và thất bại trong im lặng.

Nhóm B — Lỗi về tính đúng đắn và độ bền

#	Lỗi	Hệ quả	Cách sửa
B1	Worker mount <code>/data</code> bằng named volume	Khi triển khai thật, sandbox sẽ nhận thư mục rỗng (worker truyền đường dẫn cho Docker daemon của host , không phải của chính nó)	Chuyển sang bind mount <code>/data:/data</code>
B2	<code>--roles ""</code> (chế độ chỉ static) không hoạt động	<code>"".split(",")</code> trả về <code>[]</code> chứ không phải <code>[]</code>	Lọc bỏ phần tử rỗng
B3	Ghi file JSON lỗi trên Windows	Thiếu <code>encoding="utf-8"</code> , mặc định cp1252 không mã hóa được tiếng Việt	Chỉ định encoding tường minh
B4	Một vai lỗi làm hỏng cả lần review	<code>cli.py</code> không bắt exception theo vai như <code>worker.py</code>	Bắt lỗi từng vai, vẫn xuất báo cáo phần còn lại, trả exit code 2
B5	Gặp lỗi 429/503 của LLM là hỏng luôn	Không có retry	Bật <code>max_retries=5</code> (backoff lũy thừa)
B6	Ruff chưa từng chạy được trong sandbox	Ruff ghi cache vào <code>/src</code> đang mount read-only rồi thoát lỗi; lỗi bị nuốt	Tắt cache; thêm <code>failed_tools.txt</code> + <code>tool_status()</code> để report phân biệt "không có lỗi" với "công cụ chết"; thêm integration test chạy container thật
B7	<code>cli.py</code> gọi <code>log.error</code> khi một static tool thất bại nhưng chưa khai báo <code>log</code>	Đúng lúc cần báo lỗi thì chính dòng báo lỗi crash bằng <code>NameError</code> — cùng lớp bug "công cụ im lặng"	Khai báo logger; phát hiện khi rà code để sửa bước dedupe

Nhóm C — Tối ưu hiệu năng

Vấn đề: mỗi lần quét static mất ~350 giây. Hai nguyên nhân: - Sandbox quét **toàn bộ repo**, trong khi tầng trên chỉ giữ finding ở file có trong diff → phần lớn công sức bị vứt bỏ. - Semgrep nạp cả **2.150 rule** của mọi ngôn ngữ, kể cả khi PR chỉ sửa một file Python.

Giải pháp: chỉ quét file trong diff, và chỉ nạp rule của ngôn ngữ xuất hiện trong diff.

Kết quả đo được:

Cấu hình	Thời gian
Toàn bộ 2.150 rule (ban đầu)	349 s
Chỉ rule Python	79 s
Python + generic (đang dùng)	117 s
Dockerfile + generic (đang dùng)	71 s

→ **Nhanh gấp ~3 lần, kết quả phát hiện không đổi** (đã kiểm chứng bằng cách so sánh finding trước/sau).

Nhóm D — Sửa theo kết quả thực nghiệm (05/10)

#	Vấn đề đo được	Nguyên nhân	Cách sửa
D1	FP của cấu hình <code>both</code> (61) lớn hơn tổng FP hai nguồn (26 + 27): bước gộp không gộp được gì	Dedupe đòi <i>cùng category</i> giữa mọi nguồn, trong khi static và LLM đặt tên loại lỗi khác nhau; LLM bị dặn "không lặp lại static" nên báo thêm nhận xét phụ ngay cạnh đó (19/35 FP LLM nằm trong ±2 dòng của một finding static)	Quy tắc gộp nhận biết nguồn (mục 4, điểm 4). Thử bản "gộp theo vị trí bất kể nguồn" trước: mất 2 TP vì gộp nhầm hai vai LLM; thử bản "finding nghiêm trọng hơn làm chính": LLM tự chấm <i>high</i> cho "thiếu unit test" đề lên finding static đúng, mất 3 TP. Bản cuối không đổi nhãn, không giấu finding nặng hơn
D2	11/26 FP của <code>static</code> là bandit B101 <code>assert_used</code> trong <code>tests/</code>	<code>assert</code> là cách viết chuẩn của pytest; tài liệu bandit khuyến nghị bỏ B101 cho test code	Bộ lọc nhiễu <code>is_noise()</code> trong <code>static_tools.py</code> , có bảng luật tường minh. Nguyên tắc: chỉ thêm luật là best practice được tài liệu công cụ ghi nhận, không thêm luật vì nó làm đẹp điểm eval
D3	Chạy lại <code>both</code> thật (05/10) vẫn thấy 11 FP B101 dù D2 đã có test pass	Bộ lọc <code>is_noise()</code> được viết, có unit test riêng, nhưng chưa được nối vào <code>collect()</code> — lệnh sửa file bằng script không khớp chuỗi (file dùng CRLF) và thất bại im lặng. Rescore offline cho số đúng vì <code>rescore.py</code> gọi <code>is_noise()</code> trực tiếp, nên không ai nhận ra	Nối bộ lọc vào <code>collect()</code> ; thêm test đi qua chính <code>collect()</code> với file <code>bandit.json</code> giả; chạy lại CLI thật trên case <code>skip-test</code> xác nhận 0 B101

Hai thay đổi D1, D2 chỉ chạm bước hậu xử lý nên được kiểm chứng trước bằng `eval/rescore.py`: lấy findings thô đã lưu, chạy qua hậu xử lý mới, chấm lại (static tools tắt định; phần LLM không sinh lại). Sau đó chạy lại thật toàn bộ `both` để xác nhận (mục 7.2). D3 là bài học thứ **năm** của lớp bug "công cụ im lặng": một test cho hàm lẻ không chứng minh hàm đó được gọi; test phải đi qua đúng đường người dùng đi. Có **6 unit test** mới cho các quy tắc trên.

Nhóm E — Triển khai 7 đề xuất từ khảo sát Open Code Review (05–06/10)

Sau khi đọc mã nguồn, prompt, bộ lọc file và bài báo AACR-Bench của Open Code Review (Alibaba, Apache-2.0, >43.000 sao), đề án rút ra 7 đề xuất (`docs/DE_XUAT_CAI_TIEN_TU_OPEN_CODE_REVIEW.md`) và đã triển khai toàn bộ trong commit `d2c5a0b` . Mỗi đề xuất trả lời một điểm yếu **đã đo được** ở mục 7:

#	Đề xuất	Điểm yếu đã đo	Cách làm	Kiểm chứng
E1	Ghim nhận xét bằng đoạn code nguyên văn	Phải nới chấm điểm ± 3 dòng vì LLM đếm dòng kém	<code>Finding.existing_code</code> + <code>diff_utils.resolve_anchor</code> (mục 4, lớp 2); <code>score.py --tolerance 0</code> để chấm chặt	Unit test; 39/39 ghim bằng code ; F1 tol 0 = F1 ± 3 dòng = 89,5 % cho <code>llm</code> (trước nhóm E: 68,0 so với 70,1). Phía static chưa có cơ chế này nên <code>both</code> còn hụt 10,3 điểm ở tol 0
E2	Fact-check thay self-verify	Self-verify không có lợi ích đo được, tốn gấp đôi token	<code>agent.fact_check</code> (mục 4, lớp 3); bản cũ giữ lại là <code>--verify legacy</code> để so 3 chế độ trên cùng pipeline	Unit test cho từng căn cứ A/B; ablation 3 chế độ: fact-check loại 0 finding → <code>llm</code> và <code>llm_noverify</code> trùng nhau từng chỉ số (F1 89,5 %) nhưng fact-check tốn +40 % token ; <code>legacyverify</code> +5 FP, -1 TP . An toàn sau E8 nhưng chưa có lợi ích đo được (mục 7.6)
E3	Checklist theo loại file, có mục "Không báo"	10/35 case bị báo "thiếu unit test"; FP Dockerfile ngoài ground truth	<code>rules.py</code> + 9 file <code>app/skills/rules/*.md</code> ; SKILL.md ba vai thêm "chỉ báo vấn đề do diff gây ra"	FP của <code>llm</code> trên bộ Python giảm 27 → 6 (-78 %) mà không mất TP (giữ 34); precision 55,7 % → 85,0 %. Bộ đa ngôn ngữ: FP 13 → 6, TP giữ 18. Cải thiện lớn nhất của nhóm E (mục 7.6)
E4	Không cắt im lặng	<code>files[:25]</code> và <code>patch[:8000]</code> cắt bớt mà report vẫn như đã xem hết — lớp bug "công cụ im lặng"	<code>selection.build_batches</code> chia lô theo ngân sách ký tự (<code>max_prompt_chars</code> , <code>max_file_chars</code>); mọi lần cắt trả về và in vào mục "Phạm vi review" của report	Unit test với PR nhiều file / file lớn
E5	Chặn file bí mật và file sinh tự động trước khi gửi LLM	Rủi ro gửi <code>.env</code> , khoá riêng lên dịch vụ ngoài	<code>selection.select_for_llm</code> : cổng secret đứng trước mọi cấu hình, không bật lại được; gitleaks vẫn quét trong sandbox; dòng gitleaks đánh dấu bị che trong prompt	Unit test; đếm <code>redacted_lines</code> trong meta
E6	Nhớ nhận xét giữa các lần push	Push thêm commit thì bot đăng lại toàn bộ comment cũ	<code>postprocess.fingerprint</code> + bảng <code>posted_comments</code> ; chỉ đăng comment chưa từng đăng, kết luận check vẫn tính trên toàn bộ	Unit test; chưa thử trên GitHub thật (cần GitHub App)
E7	Nâng cấp cách đánh giá	Precision chỉ là cận dưới; chưa có PR thật	<code>judge_fp.py</code> (giám khảo LLM bỏ phiếu phân loại FP: đúng-ngoài-ground-truth / sai sự thật); <code>aacr_eval.py</code> (pilot trên PR Python thật của AACR-Bench, chấm theo vị trí + ngữ nghĩa)	Giám khảo FP: Python <code>llm</code> 6/6 và <code>both</code> 8/8 FP là "đúng nhưng ngoài ground truth", 0 bịa; đa ngôn ngữ 1/6 bịa (4,2 %). Pilot AACR trên 15 PR thật: F1 chỉ 1,1-2,6 % → bộ lỗi tiềm đánh giá quá cao (mục 7.6)

Hai việc đi kèm: (a) **gom phần LLM của CLI và worker về `pipeline.py`** để một sửa đổi không thể được nối vào đường eval mà quên đường người dùng (nguyên nhân gốc của D3); (b) `parse_output` bỏ từng finding hỏng thay vì bỏ cả lô. Có **21 unit test mới**, gồm test đi qua `run_llm_review` với backend giả để kiểm tra *thứ pipeline thực sự gửi cho LLM*, không chỉ hàm lẻ.

#	Vấn đề đo được ngay sau E2	Nguyên nhân	Cách sửa
E8	Lần chạy <code>llm</code> đầu tiên với fact-check (05/10): bước kiểm chứng loại 5 finding, trong đó 4 là finding đúng (<code>eq-none</code> ×2, <code>unused-import</code> ×2)	Model nêu căn cứ B với "bằng chứng" là chính dòng bị báo (<code>if user == None:</code> , <code>import json</code>): lý luận vòng, và phần mềm chỉ kiểm "dòng có thật trong diff" nên chấp nhận	<code>_removal_proven</code> bác căn cứ B nếu bằng chứng trùng <code>existing_code</code> của finding; test hồi quy với 3 cách chép (nguyên văn, kèm <code>+</code> , chép thiếu). Finding bị loại nay được lưu nguyên vẹn vào meta để chấm lại offline. Đã chạy lại 07/10: fact-check loại 0 finding ở cả <code>llm</code> và <code>both</code> — không còn loại oan finding đúng, xác nhận bản sửa có tác dụng (mục 7.6)
E9	Pilot PR thật (AACR): <code>both</code> sinh 286 finding cho 15 PR, trong đó 240/275 finding static nằm ở dòng không hề thay đổi của file có trong diff	Static tools quét cả file; trên dataset lỗi tiềm (file nhỏ, chỉ có lỗi được tiềm) điều này không lộ ra, trên repo thật thì phần lớn cảnh báo rơi vào code cũ. GitHub cũng từ chối đăng comment lên dòng ngoài hunk	<code>pipeline.scope_static_to_diff</code> : chỉ giữ finding static nằm trong hunk (dòng thêm mới hoặc dòng ngữ cảnh quanh đó); nối vào cả <code>cli.py</code> lẫn <code>worker.py</code> ; test. <code>both</code> trên AACR giảm 286 → 43 finding
E10	Lượt chạy pilot đầu tiên review diff đảo chiều , kèm hàng nghìn dòng không thuộc PR	<code>aacr_eval.py</code> hiểu ngược hai trường của AACR-Bench: <code>pr_source_commit</code> là base , <code>pr_target_commit</code> là head (xác minh bằng GitHub API cho cả 15 PR)	Lấy diff <code>base...head</code> qua GitHub compare; kiểm tra số dòng khớp dataset và 68/68 vấn đề chuẩn nằm trong diff trước khi chạy

E8–E10 cùng một bài học với D3: mỗi phép đo mới lại lộ ra một giả định ngầm (bằng chứng là "dòng có thật" chưa đủ; static "trong file có trong diff" chưa đủ; tên trường của dataset không tự giải thích). Cả ba được commit cùng test hồi quy trong `2133adf`.

6. Thiết kế thực nghiệm đánh giá

6.1 Bộ dữ liệu

Vì không có bộ dữ liệu chuẩn cho bài toán này, đề án **tự sinh dataset có kiểm soát**: một repo Python mẫu (nhánh `main` sạch) và **35 nhánh**, mỗi nhánh tiềm đúng **một lỗi đã biết trước**.

Loại lỗi	Số lượng	Ví dụ
Security	13	SQL injection (nối chuỗi và f-string), command injection, secret hard-code, <code>eval()</code> , <code>yaml.load</code> không an toàn, <code>pickle.loads</code> , MD5 cho mật khẩu, tắt kiểm tra TLS, Flask debug mode
DevOps	8	Base image <code>latest</code> , chạy bằng root, <code>COPY . .</code> , <code>ADD</code> thay <code>COPY</code> , <code>sudo</code> trong Dockerfile, dependency không ghim version
Bug	7	Mutable default argument, bare <code>except</code> , off-by-one, <code>== None</code> , chia cho 0, điều kiện luôn đúng, biến chưa định nghĩa
Test/QA	4	Skip test không lý do, assertion yếu, hàm mới không có test, test mất assertion
Style/Performance	4	Nối chuỗi trong vòng lặp, import thừa, <code>print()</code> thay logging, trùng lặp logic
Tổng	35 case / 36 lỗi	

Điểm mạnh về phương pháp: ground truth (file, số dòng, loại lỗi) được **tính tự động** từ nội dung file thay vì gõ tay, nên số dòng luôn chính xác kể cả khi nội dung file thay đổi.

Bộ thứ hai — đa ngôn ngữ (18 case): để kiểm tra kết quả có đúng vững khi rời Python/Dockerfile không, `make_dataset_multi.py` sinh một repo riêng gồm frontend JS/JSX/TS (XSS qua innerHTML và dangerouslySetInnerHTML, `eval`, secret, so sánh lỏng `==`, thiếu `key` trong React, thiếu `await`, assertion yếu, skip test), Go (SQL injection, bỏ qua `err`, nil dereference, thiếu test) và Java (SQL injection, MD5, resource leak). Phân bố: 7 security, 7 bug, 3 test, 1 performance.

6.2 Các cấu hình so sánh

Cấu hình	Mô tả	Mục đích
<code>static</code>	Chỉ 5 công cụ static	Đường cơ sở (baseline)
<code>llm</code>	Chỉ LLM 3 vai	Đo năng lực riêng của LLM
<code>both</code>	Kết hợp (mặc định của sản phẩm)	Kiểm chứng giả thuyết "kết hợp tốt hơn từng phần"
<code>llm_noverify</code>	Chỉ LLM, tắt bước kiểm chứng	Ablation: đo bước kiểm chứng có thật sự lọc bớt finding sai không
<code>llm_legacyverify</code>	Chỉ LLM, kiểm chứng kiểu cũ (self-verify sinh lại danh sách) trên pipeline mới	Đối chứng cho E2: so ba chế độ fact-check / legacy / off trên cùng một nền

Từ pipeline mới (nhóm E), `score.py` chấm thêm ở **tolerance 0** (đúng dòng tuyệt đối) bên cạnh ± 3 dòng, để đo tác dụng của việc ghim bằng code (E1). Hai phép đo bổ sung: `judge_fp.py` (giám khảo LLM bỏ phiếu đa số, phân loại từng FP là *đúng-nhưng-ngoài-ground-truth* hay *sai sự thật*, kèm *do diff gây ra* hay *có từ trước*) và `aacr_eval.py` (pilot trên PR Python thật ≤ 210 dòng thay đổi của AACR-Bench, chấm "trúng vị trí" ± 3 dòng và "trúng ngữ nghĩa" bằng giám khảo LLM, báo hai mức chặt/lỏng). Giới hạn phải nêu: giám khảo hiện là **cùng model** với bên sinh nhận xét nên con số "đúng" là cận trên.

6.3 Tiêu chí chấm điểm

Một phát hiện được tính **đúng (True Positive)** khi khớp với một lỗi ground truth chưa được khớp trước đó, thỏa đồng thời: - cùng file, - lệch dòng ≤ 3 , - loại lỗi (category) nằm trong danh sách chấp nhận.

Từ đó tính **Precision = TP/(TP+FP)**, **Recall = TP/(TP+FN)**, **F1**. Ngoài ra đo **thời gian/PR** và **token/PR**.

Mỗi ground truth chỉ được khớp một lần, nên việc báo trùng bị tính là dương tính giả.

Lưu ý về tính công bằng của phép đo: khi chạy thực nghiệm, ngưỡng lọc được nối ra (`MIN_SEVERITY=low` , `MIN_CONFIDENCE=0`) vì mục tiêu là đo *khả năng phát hiện*, tách biệt khỏi *chính sách lọc comment* của sản phẩm. Precision báo cáo là **cận dưới**, vì các phát hiện "đúng nhưng nằm ngoài ground truth" (ví dụ LLM chê thiếu docstring) vẫn bị tính là dương tính giả.

7. Kết quả hiện có

Thực nghiệm đã chạy **đủ 35 case × 4 cấu hình** trên bộ Python và **18 case × 3 cấu hình** trên bộ đa ngôn ngữ, không lượt nào thất bại. Ngoài ra chạy lại `11m` một lần nữa y hệt cấu hình để đo dao động.

7.1 Bộ Python — số liệu thô lúc chạy (sau khi sửa ruff, trước khi sửa bước gộp)

Cấu hình	TP	FP	FN	Precision	Recall	F1	Thời gian/PR	Token/PR
<code>static</code>	23	26	13	46,9 %	63,9 %	54,1 %	101,4 s	0
<code>11m</code>	34	27	2	55,7 %	94,4 %	70,1 %	13,3 s	7.229
<code>both</code>	35	61	1	36,5 %	97,2 %	53,0 %	119,2 s	7.552
<code>11m_noverify</code>	34	28	2	54,8 %	94,4 %	69,4 %	7,0 s	3.324
<code>11m</code> chạy lần 2	33	26	3	55,9 %	91,7 %	69,5 %	19,9 s	7.163

Tổng ground truth: 36 lỗi. Chi tiết từng case: `eval/out/results.md`.

7.2 Bộ Python — sau khi sửa bước gộp và lọc nhiễu (D1, D2), chấm lại bằng `rescore.py`

Cấu hình	TP	FP	FN	Precision	Recall	F1	So với 7.1
<code>static</code>	23	15	13	60,5 %	63,9 %	62,2 %	F1 +8,1 điểm (bỏ 11 FP B101)
<code>11m</code>	34	27	2	55,7 %	94,4 %	70,1 %	không đổi
<code>both</code>	35	34	1	50,7 %	97,2 %	66,7 %	F1 +13,7 điểm, FP 61 → 34, không mất TP nào

Chạy lại thật `both` (05/10/2026, 35/35 lượt thành công, cùng model Gemini như các lần trước):

Lần chạy	TP	FP	FN	Precision	Recall	F1	Ghi chú
Rescore offline (bảng trên)	35	34	1	50,7 %	97,2 %	66,7 %	LLM không sinh lại
Chạy thật, code lúc đó (còn bug D3)	34	44	2	43,6 %	94,4 %	59,6 %	11/44 FP là B101
Chạy thật + bộ lọc đã nối	34	33	2	50,7 %	94,4 %	66,0 %	Tương đương code sau D3 (static tắt định)

Lần chạy thật xác nhận số rescore: chênh đúng 1 TP (`skip-test` , lần này LLM bỏ sót), bằng mức dao động giữa hai lần chạy `11m` y hệt đã đo ở 7.1. FP của LLM trong `both` là 17 (rescore: 19). Số liệu `both` dùng để kết luận từ đây là **66,0 %** (chạy thật), không phải 66,7 %.

Khả năng phát hiện theo nhóm lỗi (bắt được / tổng), sau 7.2:

Nhóm	static	11m	both
security	12/13	12/13	12/13
devops	6/8	8/8	8/8
bug	4/7	6/7	7/7
test	0/4	4/4	4/4
style	1/3	3/3	3/3
performance	0/1	1/1	1/1

7.3 Bộ đa ngôn ngữ (18 case, JS/TS/Go/Java)

Cấu hình	TP	FP	FN	Precision	Recall	F1	Thời gian/PR	Token/PR
static	6	1	12	85,7 %	33,3 %	48,0 %	89,9 s	0
11m	18	13	0	58,1 %	100 %	73,5 %	33,6 s	8.336
both (thô)	18	16	0	52,9 %	100 %	69,2 %	96,2 s	8.570
both (sau D1)	18	14	0	56,2 %	100 %	72,0 %	–	–

Static chỉ bắt được 6/18 (toàn bộ là security: SQLi, eval, secret, MD5) vì bộ rule semgrep nạp sẵn cho JS/Go/Java thừa hơn Python, và không có công cụ tương đương bandit/ruff cho các ngôn ngữ này. LLM bắt 18/18.

7.4 Nhận định (pipeline trước nhóm E)

Đọc kèm mục 7.6. Các nhận định dưới đây chốt cho pipeline trước nhóm E và được giữ lại làm hồ sơ quá trình. Số liệu mới xác nhận hoặc thay thế như sau: - Điểm 1 (both recall cao nhất, F1 vẫn dưới 11m) — **vẫn đúng**: 7.6 cho 11m 89,5 so với both 87,2. - Điểm 2 (static không còn là tập con của LLM) — **vẫn đúng**. - Điểm 3 (FP còn lại chủ yếu "đúng nhưng ngoài ground truth") — **được giám khảo FP xác nhận định lượng** ở 7.6: 6/6 và 8/8 FP là hợp lệ, 0 bịa. - Điểm 4 (self-verify chưa chứng minh được lợi ích) — **kết luận mạnh hơn** ở 7.6, với ba biến thể cùng chỉ một hướng. - Toàn bộ kết luận này chỉ đúng trong điều kiện *lỗi tiêm, diff nhỏ*; pilot trên PR thật ở cuối 7.6 cho F1 1–3 %.

- 1. Giả thuyết "kết hợp tốt hơn từng phần" chỉ đúng một nửa.** both có recall cao nhất trên cả hai bộ (97,2 % và 100 %), nhưng F1 vẫn thấp hơn 11m thuần 1,5–3,4 điểm vì precision thấp hơn. Trước khi sửa D1/D2, khoảng cách là 17 điểm và FP của both **lớn hơn tổng FP hai nguồn** — tức bước gộp không hoạt động. Sau khi sửa, FP của both (34) đã **nhỏ hơn tổng** (15 + 27 = 42): bước gộp có tác dụng thật, không phải cộng gộp thô nữa.
- 2. Static không còn là tập con của LLM.** Nhận định trước đây (static không đóng góp phát hiện nào LLM bỏ sót) dựa trên số liệu khi ruff chưa chạy được. Sau khi sửa ruff, static bắt được eq-none (so sánh None bằng ==) mà LLM bỏ sót trong cả hai lần chạy, và bắt undefined-name, bare-except, mutable-default-arg tất định. Đó là lý do both đạt bug 7/7 trong khi 11m chỉ 6/7.
- 3. Phần FP còn lại của both chủ yếu là "đúng nhưng ngoài ground truth".** Rà 34 FP sau D1/D2: 5 lần dockerfile-source-not-pinned (semgrep, base image không ghim digest), 3 lần "thiếu HEALTHCHECK", 5 lần "thiếu đóng kết nối database" (base code của dataset thật sự không đóng kết nối), ~10 lần "thiếu unit test cho hàm X". Đây là nhận xét hợp lệ về mặt kỹ thuật nhưng không phải lỗi được tiêm, nên precision báo cáo là **cạn dưới** (mục 6.3).

4. Self-verify chưa chứng minh được lợi ích. Bật/tắt self-verify chênh 0,7 điểm F1 (70,1 so với 69,4), **bằng** mức dao động giữa hai lần chạy `llm` y hệt (70,1 so với 69,5), trong khi tốn gấp đôi token (7.229 so với 3.324) và gấp đôi thời gian. Nguyên nhân có thể: self-verify dùng chính model và chính vai đã sinh finding, nên ít khi "đổi ý". Đây là kết quả âm đáng báo cáo, và là hướng mở: dùng model khác hoặc vai khác để kiểm chứng.

5. Static chậm hơn LLM 3–10 lần (101 s so với 13 s trên Python; 90 s so với 34 s trên đa ngôn ngữ) do khởi tạo container và chạy tuần tự 5 công cụ, không phải do khối lượng phân tích.

6. Hai lỗi không cấu hình nào bắt được: `secret-aws` dòng 7 (artifact của cách chấm: một finding bao trùm cặp access key/secret key ở dòng 6–7, mỗi ground truth chỉ khớp một finding) và, với `llm`, `eq-none`.

7.5 Hệ quả cho thiết kế sản phẩm

- Nếu tối ưu F1 và tốc độ: `llm` thuần vẫn nhỉnh hơn trên dataset này.
- Nếu ưu tiên **không bỏ sót** (recall) và cần kết quả **tất định, kiểm toán được** cho quy trình CI: `both` sau D1/D2 là lựa chọn hợp lý, với chi phí là ~7 FP thêm trên 35 PR và 100 s/PR cho sandbox. Phần FP này gần như toàn bộ là nhận xét hợp lệ ngoài ground truth.
- Self-verify nên là tùy chọn tắt mặc định cho đến khi có bằng chứng, vì tốn gấp đôi token mà không đo được lợi ích.

Kết luận gắn với **dataset lỗi tiềm nhân tạo, diff nhỏ** — điều kiện thuận lợi cho LLM. Trên diff lớn, LLM dễ bỏ sót còn static không phụ thuộc độ dài (mục 9).

7.6 Kết quả với pipeline mới (nhóm E)

Các số ở 7.1–7.5 đo trên pipeline **trước** nhóm E và là số chốt cho phần đó. Với pipeline mới, thực nghiệm được chạy lại từ đầu vào `eval/out-v3` (Python) và `eval/out-multi-v3` (đa ngôn ngữ), cùng model `gemini-3.1-flash-lite` như các lần trước để so được. Lần chạy hoàn tất **07/10/2026: 35/35 và 18/18 lượt thành công**, không lượt nào lỗi.

Bộ Python — 35 case, 36 lỗi chuẩn

Cấu hình	TP	FP	FN	Precision	Recall	F1 (±3 dòng)	F1 (tol 0)	s/PR	token/PR	So với 7.2
<code>static</code>	21	9	15	70,0 %	58,3 %	63,6 %	54,5 %	101,4	0	F1 +1,4 điểm, nhưng mất 2 TP (ghi chú 1)
<code>llm</code> (fact-check)	34	6	2	85,0 %	94,4 %	89,5 %	89,5 %	18,5	6.355	F1 +19,4 điểm
<code>llm_noverify</code>	34	6	2	85,0 %	94,4 %	89,5 %	89,5 %	12,2	4.532	F1 +20,1 điểm so với 7.5
<code>llm_legacyverify</code>	33	11	3	75,0 %	91,7 %	82,5 %	82,5 %	18,6	8.656	thấp hơn <code>llm</code> 7,0 điểm
<code>both</code> (thô)	35	15	1	70,0 %	97,2 %	81,4 %	72,1 %	118,3	6.551	F1 +15,4 điểm
<code>both</code> (gộp static cùng dòng)	34	8	2	81,0 %	94,4 %	87,2 %	76,9 %	118,3	6.551	F1 +21,2 điểm

Dòng `both` cuối là **số chính thức** (commit `8e87c2e`, chấm lại trong `eval/out-v3-rescored`): khi static và LLM báo cùng một dòng thì gộp thành một cụm thay vì đếm hai lần. Xem ghi chú 3 về 1 TP bị mất khi gộp.

Bộ đa ngôn ngữ — 18 case (JS, React, TypeScript, Go, Java), 18 lỗi chuẩn

Cấu hình	TP	FP	FN	Precision	Recall	F1 (±3 dòng)	F1 (tol 0)	s/PR	token/PR	So với 7.3
11m	18	6	0	75,0 %	100 %	85,7 %	76,2 %	18,3	6.994	F1 +12,2 điểm
both	18	8	0	69,2 %	100 %	81,8 %	72,7 %	109,7	7.052	F1 +9,8 điểm

Chấm lại theo cụm không làm đổi số của bộ này.

Trả lời ba câu hỏi đặt ra cho bảng

(1) E3 — checklist "Không báo" có giảm FP mà không mất TP? Có, và đây là cải thiện lớn nhất của nhóm E. Trên bộ Python, FP của 11m giảm 27 → 6 (−78 %) trong khi TP giữ nguyên 34. Precision nhảy từ 55,7 % lên 85,0 %. Bộ đa ngôn ngữ cũng vậy: FP 13 → 6, TP giữ 18. Nói cách khác, phần lớn "báo nhầm" trước đây không phải do model yếu mà do thiếu định nghĩa phạm vi cho nó.

(2) Fact-check (sau khi sửa E8) có loại được FP nào? Không. Nó loại đúng 0 finding ở cả 11m và both. Hệ quả là 11m và 11m_noverify trùng nhau từng chỉ số (34/6/2 · 85,0 % · 94,4 % · F1 89,5 %), chỉ khác chi phí:

	11m_noverify	11m (fact-check)	Chênh
Token/PR	4.532	6.355	+40 %
Thời gian/PR	12,2 s	18,5 s	+52 %
F1	89,5 %	89,5 %	0

Sau khi sửa lỗi lý luận vòng (E8), fact-check an toàn — không còn loại oan finding đúng như lần chạy 05/10 — nhưng chưa chứng minh được lợi ích. Thiết kế cũ còn tệ hơn hẳn: 11m_legacyverify làm tăng 5 FP, mất 1 TP, và tốn gấp 1,9 lần token so với noverify (8.656 so với 4.532). Kết luận âm về bước tự kiểm chứng ở mục 7.5 nay rõ hơn trước, với ba biến thể cùng chỉ về một hướng.

(3) E1 — ghim bằng đoạn code nguyên văn có kéo F1 ở tolerance 0 về gần F1 ở ±3 dòng? Có, với nhận xét của LLM: 39/39 finding của 11m ghim được bằng code, và F1 ở tolerance 0 bằng đúng F1 ở ±3 dòng (89,5 % cả hai) — nghĩa là mọi nhận xét trở đúng dòng tuyệt đối, không còn phải nói sai số. Trước nhóm E, F1 tol 0 của 11m là 68,0 so với 70,1 ở ±3 dòng.

Nhưng với both thì khoảng cách vẫn còn: 87,2 % so với 76,9 % (hụt 10,3 điểm). Lý do là phía static không có cơ chế ghim này — finding của semgrep/bandit/ruff vẫn chỉ mang số dòng. Đây là việc còn lại của E1.

Giám khảo phân loại FP (judge_fp.py , gemini-3.5-flash-lite , 3 phiếu mỗi FP)

Cấu hình	FP	Đúng sự thật (ngoài ground truth)	Sai sự thật (bịa)	Tỷ lệ bịa / tổng nhận xét
Python 11m	6	6 (1 là vấn đề có sẵn, không do diff)	0	0,0 %
Python 11m_noverify	6	6	0	0,0 %
Python both (gộp cụm)	8	8 (static 4, LLM 4; 2 có sẵn)	0	0,0 %
Đa ngôn ngữ 11m	6	5	1	4,2 %

Đọc theo cách này thì precision "nếu tính cả nhận xét đúng nằm ngoài ground truth" là 100 % cho cả hai cấu hình Python và 95,8 % cho bộ đa ngôn ngữ. Tức là ở bộ lỗi tiềm, hệ thống hầu như không bịa; phần lớn cái bị chấm là "sai" thực chất là nhận xét hợp lệ mà ground truth không liệt kê.

Cảnh báo về chính phép đo này: giám khảo hạng nhẹ có xu hướng dễ đồng ý. Thử lại một mẫu bằng gemini-3.8-flash (model mạnh hơn, chỉ 20 request/ngày nên không chạy được cả bộ) thì nó bỏ 2/3 phiếu "invalid" cho nhận xét "thiếu test cho hàm ping" — tức mẫu đó đáng bị tính là bịa. Nên con số "0 % bịa" là cận trên lạc quan, cần giám khảo mạnh hơn hoặc rà tay để chốt.

Ghi chú về độ sạch của số liệu

- 1. `static` được chấm lại, không chạy lại. Static là tắt định nên thay vì chạy lại container, lần chạy gốc (`eval/out/static`) được đưa qua **hậu xử lý hiện tại** — lọc nhiễu B101 và gộp mọi cảnh báo static cùng dòng thành một cụm — rồi chấm lại vào `eval/out-v3-rescored/static` . Về kết quả, việc này tương đương chạy lại. Đáng chú ý: FP giảm **15 → 9** nhưng **mất 2 TP** (23 → 21), vì nhãn loại lỗi của cụm lấy theo cảnh báo chính — đúng hạn chế đã nêu ở ghi chú 3 cho `both` . Ròng lại F1 chỉ +1,4 điểm so với 7.2, nên việc gộp cụm **có cái giá của nó**, không phải cải thiện thuần.
- 2. **Giám khảo FP đã chạy lại cho `llm_noverify`** sau khi case cuối hoàn tất: TP 34, FP 6, **6/6 hợp lệ, 0 bịa** — trùng với `llm` , đúng như dự đoán vì fact-check không loại finding nào. (Lần chấm trước thực hiện khi cấu hình này mới xong 34/35 case nên thiếu 1 FP.)
- 3. `both` **gộp cụm mất 1 TP** (`docker-apt-no-clean`): khi gộp nhiều finding cùng dòng thành một cụm, nhãn loại lỗi của cụm lấy theo finding chính, nên cụm đó không còn khớp loại của ground truth. Đây là hạn chế của cách gộp, không phải mất khả năng phát hiện — lỗi vẫn được báo đúng dòng.
- 4. **Tolerance 0 trên bộ đa ngôn ngữ mất 2 TP vì khác định nghĩa dòng**, không phải vì ghim sai: nhận xét ghim ở đầu vòng lặp (dòng 17) còn đáp án ghi dòng 20; trường hợp nil-deref ghim dòng 21 còn đáp án dòng 22.

Pilot trên PR thật — AACR-Bench, tập con Python (đã chốt 06/10, `eval/aacr/out/results.md`). 15 PR ≤ 210 dòng thay đổi từ ComfyUI, browser-use, ragflow, vllm, langflow; **68 vấn đề chuẩn** do kỹ sư xác nhận. Model review và giám khảo: `gemini-3.5-flash-lite` , giám khảo bỏ 3 phiếu cho mỗi cặp (finding, vấn đề chuẩn). *Chặt* = cùng file, dòng trong ±3, giám khảo xác nhận cùng vấn đề; *lỏng* = bỏ điều kiện dòng.

Cấu hình	Finding	TP (chặt)	Precision	Recall	F1	TP (lỏng)
<code>llm</code>	8	1	12,5 %	1,5 %	2,6 %	1
<code>both</code> (static chưa lọc theo hunk)	286	2	0,7 %	2,9 %	1,1 %	2
<code>both</code> + lọc static theo hunk (E9)	43	1	2,3 %	1,5 %	1,8 %	1

Theo mức ngữ cảnh cần có để thấy vấn đề (phân loại của AACR-Bench): 21 vấn đề *diff-level* — bắt được 1; 29 *file-level* — 0 (1 với `both` chưa lọc, nhờ một finding static ngoài hunk); 18 *repo-level* — 0.

Đọc kết quả này thế nào. Đây là kết quả âm và phải báo cáo thẳng:

- 1. **Bộ eval lỗi tiềm đánh giá quá cao.** Cùng pipeline, F1 trên lỗi tiềm ở mức 86–90 (sơ bộ) nhưng trên PR thật là 1–3. Lỗi tiềm là lỗi "sách giáo khoa" nằm trọn trong diff nhỏ; vấn đề thật của AACR phần lớn cần ngữ cảnh file hoặc repo (47/68), và ngay 21 vấn đề diff-level cũng không phải mẫu lỗi quen thuộc. Để so: trong bài báo AACR-Bench, agent tốt nhất của chính Alibaba đạt recall khoảng 10 % trên bộ đầy đủ.
- 2. **LLM gần như im lặng trên code thật:** 8 nhận xét cho 15 PR, 33 lượt gọi, không vai nào lỗi, fact-check không loại gì. Recall thấp vì model (hạng nhẹ) và checklist bảo thủ ("chỉ báo khi chắc"), **không** phải do lớp lọc loại mất. Đây là mặt trái của E3: checklist giảm FP trên bộ lỗi tiềm nhưng cũng làm model ngại báo trên code lạ.
- 3. **Static trên repo thật chủ yếu báo code cũ** (E9): 240/275 finding ngoài hunk. Sau khi lọc theo hunk, `both` còn 43 finding, precision vẫn chỉ 2,3 % — 42 finding còn lại cần được `judge_fp.py` phân loại (đúng-ngoài-ground-truth hay sai) trước khi kết luận; ground truth của PR thật không bao giờ đầy đủ nên precision ở đây là cận dưới.
- 4. **Cỡ mẫu 15 PR và 1–2 TP** không cho phép so `llm` với `both` ; pilot này chỉ xác lập *mức* (order of magnitude) và chỉ ra bộ eval nào đáng tin hơn.

Hệ quả cho báo cáo: kết luận ở 7.4–7.5 chỉ đúng trong điều kiện "diff nhỏ, lỗi nằm trọn trong diff"; phần hướng phát triển (mục 9) phải thêm *ngữ cảnh file/repo cho LLM* (AACR cho thấy 69 % vấn đề thật cần nó) và *so sánh model mạnh hơn* trên cùng 15 PR.

Kiểm thử

`pytest` (06/10, Docker bật): **39/39 pass** — gồm 10 test pipeline cũ (parse diff, router, quy tắc dedupe, lọc nhiễu bandit, hồi quy self-verify), 22 test cho nhóm E (trong đó có test đi qua `run_llm_review` với backend giả, và test hồi quy E8) và các integration test chạy container thật (tự bỏ qua khi không có Docker).

8. Trạng thái các hạng mục

Hạng mục	Trạng thái
Kiến trúc và toàn bộ mã nguồn	✅ Hoàn thành (~1.700 dòng)
Sandbox static analysis (5 công cụ)	✅ Đã chạy và kiểm chứng
Agent 3 vai + self-verify	✅ Đã chạy đầu-cuối với Google Gemini
CLI chạy offline	✅ Hoàn thành
Bộ thực nghiệm (dataset + chấm điểm)	✅ Hoàn thành — bộ Python 35 case + bộ đa ngôn ngữ 18 case
Kiểm thử đơn vị / tích hợp	✅ 8/8 unit pass; 8 integration test (cần Docker)
Chạy thực nghiệm đầy đủ	✅ Hoàn thành — $35 \times 4 + 18 \times 3 + 35$ (lần 2) = 229 lượt, không lượt nào thất bại
Sửa bước hợp nhất static/LLM theo kết quả đo	✅ Xong 05/10 (D1, D2), kiểm chứng offline bằng <code>rescore.py</code>
Chạy lại <code>both</code> thật với hậu xử lý mới	✅ Xong 05/10 — 35/35 lượt, F1 66,0 % sau khi nối bộ lọc; phát hiện và sửa bug D3
Khảo sát Open Code Review và triển khai 7 đề xuất (nhóm E)	✅ Xong 05/10 (commit <code>d2c5a0b</code>), 21 unit test mới; sửa E8–E10 ngày 06/10 (commit <code>2133adf</code>)
Chạy lại thực nghiệm với pipeline mới (<code>eval/out-v3</code>)	🔄 Đang chạy — dừng vì hạn mức Gemini; tiếp tục 07/10 (mục 7.6)
Pilot trên PR thật (AACR-Bench)	✅ Xong 06/10 — 15 PR, kết quả âm (F1 1–3 %), đã phân tích ở 7.6
Triển khai GitHub App thật	⌚ Chưa — cần đăng ký GitHub App và tunnel công khai; dấu vân tay comment (E6) chỉ mới có unit test
Viết báo cáo/luận văn	🔄 Đang viết — phần chữ cho nhóm E đã có, chờ bảng số 7.6

9. Hạn chế và hướng phát triển

Hạn chế hiện tại (nên chủ động nêu khi bảo vệ)

- Dataset là lỗi tiềm nhân tạo**, không phải PR thật từ dự án mở. Lỗi tiềm thường "sạch" và dễ nhận ra hơn lỗi thực tế, nên kết quả có thể lạc quan hơn thực tế.
- Cỡ mẫu 35 case** đủ để so sánh xu hướng nhưng chưa đủ để kết luận có ý nghĩa thống kê.
- Bộ đa ngôn ngữ còn nhỏ (18 case)** và static tools cho JS/Go/Java chỉ có semgrep, nên so sánh static/LLM ở đó nghiêng về LLM.

4. **Phụ thuộc vào một nhà cung cấp LLM**; kết quả có thể khác khi đổi model. (Kiến trúc đã tách qua `AgentBackend` protocol nên đổi model không phải sửa logic.)
5. **Bộ nhớ repo chưa có vòng phản hồi tự động**: hiện phải nhập tay, chưa học từ phản ứng 👍 / 🗨️ của lập trình viên.
6. **Chưa đo chi phí thực tế bằng tiền** trên quy mô lớn.
7. **Lần chạy lại thật** `both` **vẫn có prompt LLM chứa tóm tắt static có B101** (bug D3 được phát hiện nhờ chính lần chạy đó). Static tắt định nên lọc lại là tương đương, nhưng phần LLM chưa được chạy với tóm tắt đã lọc; chênh lệch kỳ vọng nhỏ hơn mức dao động giữa hai lần chạy.
8. **Cách chấm điểm phạt oan trường hợp một phát hiện bao trùm nhiều ground truth liên nhau** (xem case `secret-aws`, mục 7.4), làm recall báo cáo thấp hơn thực tế.
9. **Self-verify dùng chính model đã sinh finding** nên chưa đo được lợi ích (mục 7.4, điểm 4). Fact-check (E2) đổi cách hỏi và thêm kiểm chứng cơ học, nhưng người đề nghị loại vẫn là cùng model; E8 cho thấy model hạng nhẹ vẫn tìm được cách "lách" (lý luận vòng) cho tới khi phần mềm chặn.
10. **Nguy cơ overfit checklist vào bộ eval tự tạo**. Các mục "Không báo" (E3) được viết sau khi đã nhìn thấy FP của bộ eval (thiếu HEALTHCHECK, thiếu đóng kết nối, thiếu unit test). Giảm FP trên *cùng* bộ đó vì thế không chứng minh checklist tổng quát. Hai biện pháp giảm thiểu: chỉ đưa vào checklist những mục có căn cứ ngoài bộ eval (tài liệu công cụ, `rule_docs` của Open Code Review), và đo lại trên PR thật (pilot AACR, E7). Khi bảo vệ nên nói rõ điều này thay vì để hội đồng hỏi.
11. **Giám khảo LLM trong** `judge_fp.py` **và** `aacr_eval.py` **là cùng model với bên sinh nhận xét**, nên tỉ lệ FP được phân loại "đúng nhưng ngoài ground truth" là cận trên; muốn chặt hơn phải dùng model khác hoặc rà tay một mẫu.
12. **Hạn mức LLM free tier (500 request/ngày/model) chỉ phôi tiến độ thực nghiệm**: một vòng đo đầy đủ mất hơn một ngày và không chạy song song được; đổi model để chạy nhanh hơn thì mất khả năng so sánh với số cũ.
13. **Dấu vân tay comment (E6) và mục "Phạm vi review" chỉ mới có unit test**, chưa chạy trên GitHub thật với nhiều lần push.
14. **Trên PR thật hệ thống gần như không bắt được gì** (pilot AACR, 7.6): recall 1,5–2,9 % trên 68 vấn đề do kỹ sư xác nhận. Hai nguyên nhân đo được: 69 % vấn đề cần ngữ cảnh file/repo mà pipeline chỉ đưa diff + 30 dòng quanh đó; model hạng nhẹ cộng checklist bảo thủ nên LLM im lặng. Đây là hạn chế lớn nhất và nên là câu đầu tiên khi nói về kết quả.

Hướng phát triển

- **Ngữ cảnh file/repo cho LLM** (ưu tiên cao nhất sau pilot AACR): đưa định nghĩa hàm/lớp được gọi trong diff, hoặc để agent được hỏi thêm file, thay vì chỉ 30 dòng quanh hunk.
 - **Đo cùng 15 PR AACR với model mạnh hơn** (backend Anthropic đã có) để tách "do model" khỏi "do pipeline".
 - Hỗ trợ GitLab/Bitbucket; suggestion block (cho phép nhấn "Apply" ngay trên GitHub).
 - Dashboard web theo dõi chất lượng code theo thời gian.
 - Vòng phản hồi 👍 / 🗨️ tự động cập nhật bộ nhớ repo.
 - Đa người thuê (multi-tenant) và tính phí — hướng sản phẩm micro-SaaS.
-

10. Dự kiến câu hỏi phản biện và hướng trả lời

H: Thực nghiệm cho thấy `both` kém hơn `11m` về F1. Vậy phần static analysis có còn ý nghĩa không? Đ: Em báo cáo đúng như đo được: F1 của `both` thấp hơn `11m` 1,5–3,4 điểm. Nhưng có bốn điểm. (1) `both` có **recall cao nhất** trên cả hai bộ, và static bắt được lỗi LLM bỏ sót trong cả hai lần chạy (`eq-none`), nên static không phải tập con của LLM. (2) Khoảng cách ban đầu là 17 điểm, do bước gộp đòi cùng category nên không gộp được gì — thực nghiệm chỉ ra lỗi này và em đã sửa, sau đó FP của `both` nhỏ hơn tổng FP hai nguồn. (3) Phần FP còn lại chủ yếu là nhận xét hợp lệ ngoài ground truth (base image không ghim digest, thiếu đóng kết nối). (4) Static **tất định và kiểm toán được** — cùng đầu vào luôn cho cùng đầu ra, trong khi hai lần chạy `11m` y hệt đã lệch nhau 1 TP; đây là yêu cầu bắt buộc ở nhiều quy trình CI. Kết luận gắn với dataset lỗi tiềm, diff nhỏ — điều kiện thuận lợi cho LLM.

H: Self-verify — đóng góp chính — có tác dụng không? Đ: Ban đầu thì không. Em làm ablation: tắt self-verify thì F1 giảm 0,7 điểm, bằng đúng mức dao động giữa hai lần chạy LLM y hệt, trong khi tốn gấp đôi token. Em báo cáo đây là kết quả âm, rồi thay bằng fact-check (mục 4, lớp 3): model không được sinh lại danh sách, chỉ được đề nghị loại khi có bằng chứng, và phần mềm kiểm bằng chứng đó. Lần chạy đầu với fact-check lại lộ ra một lỗi mới — model lấy chính dòng bị báo làm "bằng chứng" (E8) — và em chặn bằng code. Kết quả sau khi chặn (07/10): fact-check không loại oan finding nào nữa, nhưng cũng **không loại được FP nào** — `11m` và `11m_noverify` ra kết quả y hệt (F1 89,5 %) mà fact-check tốn thêm 40 % token. Biến thể cũ (`legacyverify`) thì làm xấu đi: thêm 5 FP, mất 1 TP. Điểm em muốn nhấn: mỗi lớp chống bịa chỉ đáng tin khi có một phép đo đối chứng đi kèm, và phép đo đã hai lần chỉ ra lớp này chưa làm đúng việc.

H: Trên PR thật hệ thống đạt F1 1–3 %. Vậy toàn bộ số 60–90 % ở trên có ý nghĩa gì? Đ: Em chủ động chạy pilot đó để biết số lỗi-tiềm đáng tin tới đâu, và câu trả lời là: không nhiều. Bộ lỗi tiềm đo được một thứ hẹp — "khi lỗi quen thuộc nằm trọn trong diff nhỏ, pipeline có bắt và có bịa không" — và ở phạm vi đó các kết luận (static bổ sung cho LLM, bước gộp, fact-check) vẫn đúng. Nhưng 69 % vấn đề thật cần ngữ cảnh file/repo mà pipeline không đưa cho model, và model hạng nhẹ gần như im lặng trên code lạ. Em ghi đây là hạn chế số một và hướng phát triển số một. Để tham chiếu, agent tốt nhất trong bài báo AACR-Bench cũng chỉ đạt recall khoảng 10 %, nên bài toán thật khó hơn nhiều so với bộ eval tự tạo.

H: Checklist "Không báo" được viết sau khi nhìn thấy lỗi của bộ eval — có phải đang "gọt" hệ thống cho vừa bộ đề không? Đ: Đúng là có rủi ro đó và em ghi rõ ở mục 9. Hai cách em hạn chế: chỉ đưa vào checklist những mục có căn cứ ngoài bộ eval (tài liệu công cụ, bộ `rule_docs` mà Alibaba dùng hai năm nội bộ), và đo lại trên PR thật của AACR-Bench — bộ này không ai trong nhóm nhìn thấy khi viết checklist. Nếu FP giảm trên bộ tự tạo mà không giảm trên PR thật thì checklist chỉ là overfit, và em sẽ báo cáo như vậy.

H: Tại sao không dùng thẳng ChatGPT/Copilot để review? Đ: Ba lý do. (1) *An toàn*: gửi toàn bộ mã nguồn cho dịch vụ ngoài là rủi ro với repo nội bộ; kiến trúc này cho phép trở về model chạy nội bộ mà không sửa logic. (2) *Độ tin cậy*: LLM đơn lẻ hallucinate; hệ thống có sáu lớp kiểm chứng, bốn lớp là phần mềm tất định. (3) *Tích hợp quy trình*: tự động chạy khi mở PR, comment đúng dòng, cập nhật trạng thái Checks để chặn merge — không cần thao tác thủ công.

H: Chia ba vai có thực sự cần thiết không, hay chỉ là chia prompt cho vui? Đ: Hai lý do kỹ thuật. (1) *Giảm nhiễu*: mỗi vai có checklist và thang severity riêng, tránh tình trạng một prompt khổng lồ khiến LLM trả lời hơi hợt ở mọi khía cạnh. (2) *Tiết kiệm chi phí*: router chỉ kích hoạt vai cần thiết — PR chỉ sửa Dockerfile thì không gọi vai QA. Việc chứng minh định lượng lợi ích của kiến trúc đa vai là một hướng thực nghiệm mở rộng.

H: Làm sao biết LLM không bịa lỗi? Đ: Sáu lớp (mục 4), bốn lớp là phần mềm tất định: ghim nhận xét bằng đoạn code chép nguyên văn (không tin số dòng), fact-check có kiểm chứng cơ học, `commentable_line()` (dòng phải nằm trong diff), dedupe ưu tiên static tool; hai lớp còn lại là ràng buộc trong prompt và checklist "Không

báo". Ngoài ra, thực nghiệm đo precision chính là để định lượng tỷ lệ bịa này, và `judge_fp.py` tách FP "đúng nhưng ngoài đáp án" khỏi FP "sai sự thật".

H: Nếu ai đó mở PR chứa mã độc thì sao? Đ: Mã trong PR **không bao giờ được thực thi**. Chỉ các công cụ static đọc file, trong container không có mạng, bỏ toàn bộ capability, source mount chỉ đọc, giới hạn CPU/RAM/số tiến trình. Kể cả khi một công cụ static bị khai thác, nó không có đường ra mạng.

H: Tại sao precision chỉ khoảng 39–56%? Đ: Cần phân biệt hai loại dương tính giả. Loại thứ nhất là **bịa thật sự** — đây là vấn đề. Loại thứ hai là **phát hiện đúng nhưng nằm ngoài ground truth**: ví dụ khi tiêm lỗi SQL injection, công cụ cũng báo thêm "thiếu xử lý ngoại lệ" — đúng về mặt kỹ thuật nhưng không nằm trong danh sách lỗi đã tiêm. Cách chấm hiện tại tính cả hai là sai, nên precision báo cáo là **cận dưới**. Bước tiếp theo là rà thủ công để tách hai loại này.

H: Chi phí vận hành bao nhiêu? Đ: Static tools miễn phí. Phần LLM đo được khoảng 7.000 token/PR ở cấu hình chỉ LLM. Với model hạng nhẹ, chi phí ở mức vài cent mỗi PR. Đây là một chỉ số được đo trong thực nghiệm.

H: Em đã tự kiểm chứng hệ thống chạy đúng chưa, hay chỉ viết xong rồi để đó? Đ: Quá trình chạy thử phát hiện **10 lỗi thật** (mục 5), trong đó có ba lỗi khiến chức năng **hoàn toàn không hoạt động mà vẫn không báo lỗi**: semgrep chưa từng chạy, bandit chưa từng được gọi, và bước self-verify luôn thất bại trong im lặng. Nếu chỉ đọc code mà không chạy thực tế thì không thể phát hiện được những lỗi này.

11. Thuật ngữ

Thuật ngữ	Giải thích
Hallucination	LLM tạo ra thông tin nghe hợp lý nhưng không đúng thực tế — ở đây là bịa ra lỗi không tồn tại
Static analysis	Phân tích mã nguồn mà không chạy chương trình
Precision / Recall	Precision: trong những gì báo, bao nhiêu phần đúng. Recall: trong những lỗi có thật, bắt được bao nhiêu phần
F1	Trung bình điều hòa của precision và recall — chỉ cao khi cả hai đều cao
Ground truth	Đáp án đúng đã biết trước, dùng làm chuẩn để chấm điểm
Sandbox	Môi trường cô lập để chạy thứ không đáng tin cậy
Unified diff	Định dạng chuẩn biểu diễn thay đổi giữa hai phiên bản file
Webhook	Cơ chế để dịch vụ ngoài (GitHub) chủ động gọi vào hệ thống khi có sự kiện
HMAC	Mã xác thực thông điệp dùng khóa bí mật — chứng minh webhook đến từ GitHub
Idempotency	Tính chất gọi nhiều lần cho kết quả như gọi một lần
Self-verify	Bước cho LLM tự kiểm chứng lại kết quả của chính mình (bản đầu; đã thay bằng fact-check)
Fact-check	Bước kiểm chứng mới: mặc định giữ mọi nhận xét; LLM chỉ được đề nghị loại khi nêu bằng chứng, và phần mềm kiểm bằng chứng đó trên diff
Ghim (anchor)	Tìm đúng dòng của một nhận xét bằng cách so khớp đoạn code LLM chép nguyên văn, thay vì tin số dòng LLM đếm
Checklist "Không báo"	Danh sách theo loại file nói rõ điều gì <i>không</i> phải lỗi do thay đổi này gây ra, để giảm báo động giả
Dấu vân tay (fingerprint)	Mã nhận dạng ổn định của một nhận xét (file + loại + nội dung dòng code) để không đăng lại nhận xét cũ khi push thêm
Tolerance 0 / ±3 dòng	Chấm điểm đòi đúng dòng tuyệt đối / cho phép lệch tối đa 3 dòng
AACR-Bench	Bộ PR thật kèm nhận xét đã được kỹ sư xác nhận, do Alibaba công bố cùng Open Code Review